

מטלה 2

אלגוריתמי Actor-Critic בסביבת Level-Based Foraging

הגשה בקבוצות של עד 3 סטודנטים

מטרת המטלה

במטלה זו נעבוד עם ה-codebase הרשמי המלווה את ספר הקורס, המכיל מימושים פשוטים וקריאים של אלגוריתמי MARL עמוקים. נאמן סוכני Independent A2C (IA2C) בסביבת level-based foraging, נבחן את השפעת ההיפר-פרמטרים, ונשווה בין critic עצמאי ל-critic מרכזי בסביבת המחסן הרבובטי rware. הערה: טריקים של מימוש, הנחוצים לעיתים לביצועים מיטביים, כמעט שאינם קיימים ב-codebase כדי לשמור על פשטות הקוד — חלק מהביצועים הוקרב לטובת קריאות.

הכנת הסביבה

1. שכפלו את ה-codebase של הספר כ-submodule בתוך ריפו התרגילים באמצעות הפקודה:

```
git submodule update --init --recursive
```

2. נווטו לתיקייה marl-book-codebase והתקינו את התלויות ואת החבילה עצמה:

```
cd marl-book-codebase pip install -r requirements.txt pip install -e .
```

3. הכירו את מבנה ה-codebase (מומלץ לקרוא את README.md):

- marlbase/run.py — הסקריפט הראשי להפעלת ריצות אימון
- marlbase/search.py — סקריפט להרצות מרובות, מיועד לחיפוש היפר-פרמטרים
- marlbase/ac — מימושי אלגוריתמי ה-actor-critic (A2C ו-PPO), עם critic עצמאי או critic מרכזי המותנה בתצפיות המשותפות של כל הסוכנים
- marlbase/dqn — מימוש DQN והרחבות פירוק הערך VDN ו-QMIX
- marlbase/configs — קובצי קונפיגורציה: default.yaml עם ברירות המחדל, וקובץ היפר-פרמטרים לכל אלגוריתם תחת algorithm

4. התקינו את סביבת level-based foraging:

```
pip install git+https://github.com/uoe-agents/lb-foraging.git
```

5. בדקו שהכול עובד: נווטו לתיקייה marlbase והריצו אימון קצר; אמורה להופיע התקדמות אימון סדירה בקונסול:

```
python run.py +algorithm=ia2c env.name="lbforaging:Foraging-8x8-3p-2f-v3"
env.time_limit=50 algorithm.total_steps=100000
```

אימון סוכני IA2C

IA2C הוא ההרחבה מרובת הסוכנים של A2C, שבה כל סוכן לומד מדיניות ופונקציית ערך באופן עצמאי. הריצו אימון מלא במשימת level-based foraging:

```
python run.py +algorithm=ia2c env.name="lbforaging:Foraging-8x8-3p-2f-v3"
env.time_limit=50 algorithm.total_steps=1000000 algorithm.name=ia2c
env.standardise_rewards=True algorithm.video_interval=200000 seed=0
```

הפקודה מאמנת את הסוכנים, מעריכה אותם באופן תקופתי ומפיקה סרטוני rollout. בסיום, אתרו את תיקיית התוצאות תחת `outputs/lbforaging:Foraging-8x8-3p-2f-v3/ia2c/...` — המדדים נשמרים בקובץ `results.csv` והסרטונים בתיקיית `videos`. צפו בכמה סרטונים כדי לראות את השתפרות המדיניות. ניתן לצייר את התשואות לאורך האימון:

```
python utils/postprocessing/plot_runs.py --source outputs
```

ניתן להציג גם מדדים אחרים מתוך `results.csv`, למשל את אנטרופיית המדיניות:

```
python utils/postprocessing/plot_runs.py --source outputs --metric entropy
```

ניסויים בהיפר-פרמטרים

שנו היפר-פרמטרים של האלגוריתם ובחנו את ההשפעה על האימון — למשל קצב הלמידה, מקדם גולריזציית האנטרופיה או ארכיטקטורת הרשת. ניתן להריץ כמה ריצות במקביל באמצעות `multirun`:

```
python run.py -m +algorithm=ia2c env.name="lbforaging:Foraging-8x8-3p-2f-v3"
env.time_limit=50 algorithm.total_steps=1000000 env.standardise_rewards=True
seed=0 algorithm.entropy_coef=0.001,0.01,0.1
```

הפקודה מריצה שלוש ריצות אימון עם מקדמי אנטרופיה שונים (משך הריצה — דקות ספורות, בהתאם לחומרה). כל התוצאות נשמרות תחת `multirun`. ליהוי ההיפר-פרמטרים הטובים ביותר ולציור התשואות של כל התצורות:

```
python utils/postprocessing/find_best_hyperparams.py --source multirun python
utils/postprocessing/plot_runs.py --source multirun
```

השוואה ל-critics מרכזיים

נשווה את IA2C לסוכנים המאומנים באותו אלגוריתם בסיסי אך עם critic מרכזי המותנה בתצפיות המשותפות של כל הסוכנים. כמשימה מאתגרת נשתמש בסביבת המחסן הרבובטי. התקינו אותה:

```
pip install git+https://github.com/uoe-agents/robotic-warehouse.git
```

אימון מלא של IA2C ו-MAA2C במשימה `rware-tiny-4ag-v2` היה נעשה כך:

```
python run.py -m +algorithm=ia2c,maa2c env.name="rware:rware-tiny-4ag-v2"
env.time_limit=500 algorithm.total_steps=20000000
env.standardise_rewards=True seed=0,1,2
```

אולם מדובר בשש ריצות אימון של 20,000,000 צעדים כל אחת – זמן ריצה ארוך מאוד שאינו ריאלי בהכרח. במקום זאת השתמשו בנתוני האימון המוכנים שבתיקייה `deep_marl_data/rware_tiny_4ag`, הכוללים את IA2C, MAA2C, IPPO ו-MAPPO. השוו את ביצועי האלגוריתמים ובחנו מדדים כגון התשואות (mean_episode_returns), אנטרופיית המדיניות (entropy) והפסדי ה-actor (actor_loss) וה-critic (value_loss):

```
python utils/postprocessing/plot_runs.py --source
path/to/deep_marl_data/rware_tiny_4ag --metric ...
```

ניתן גם לצפות בסרטוני ה-rollout שנשמרו לאורך האימון בתיקיית videos של כל ריצה, או לטעון checkpoint של מודל ולהפיק ממנו סרטון:

```
python eval.py path=path/to/deep_marl_data/rware_tiny_4ag/...
env.name="rware:rware-tiny-4ag-v2" env.time_limit=500
```

כברירת מחדל נטען ה-checkpoint האחרון של הריצה; ניתן לבחור נקודת זמן אחרת באמצעות הארגומנט `load_step` (רשימת ה-checkpoints הזמינים נמצאת בתיקיית checkpoints של הריצה). הערה: לריצות שבתיקיית `deep_marl_data` בוצע כוונן היפר-פרמטרים מינימלי בלבד, ולכן הביצועים והדירוג היחסי בין האלגוריתמים אינם בהכרח משקפים את יכולותיהם תחת כוונן מלא.

Assignment 2

Actor-Critic Algorithms in Level-Based Foraging

Submission in groups of up to 3 students

Goal

In this assignment we will make use of the official textbook codebase, which contains easy-to-understand implementations of deep MARL algorithms. We will train Independent A2C (IA2C) agents in a level-based foraging task, experiment with hyperparameters, and compare independent critics to centralised critics in the robotic warehouse (rware) environment.

Note: implementation tricks, while necessary for some algorithms to perform well, are sparse in this codebase so as not to complicate the code — some performance has been sacrificed for simplicity.

Setup

1. Clone the marl-book codebase within this repository as a submodule by running:

```
git submodule update --init --recursive
```

2. Navigate to the `marl-book-codebase` directory and install the required dependencies and the codebase itself:

```
cd marl-book-codebase pip install -r requirements.txt pip install -e .
```

3. Familiarise yourself with the codebase (see also the README.md file):
 - `marlbase/run.py` — the main script to start training runs
 - `marlbase/search.py` — the script to initiate multiple training runs, designed for hyperparameter search
 - `marlbase/ac` — actor-critic algorithms (A2C and PPO), trained independently or with centralised critics conditioned on the joint observations of all agents
 - `marlbase/dqn` — the DQN algorithm and the value decomposition extensions VDN and QMIX
 - `marlbase/configs` — configuration files: `default.yaml` with default hyperparameters, and per-algorithm files under `algorithm`
4. Install the level-based foraging environment:

```
pip install git+https://github.com/uoe-agents/lb-foraging.git
```

5. Test that everything works: navigate to the `marlbase` directory and run a short training run; you should see regular training progress printed to the console:

```
python run.py +algorithm=ia2c env.name="lbforaging:Foraging-8x8-3p-2f-v3"  
env.time_limit=50 algorithm.total_steps=100000
```

Train Agents with Independent Advantage Actor-Critic (IA2C)

IA2C is the multi-agent extension of advantage actor-critic (A2C), where each agent learns its policy and value function independently. Start a full training run in the level-based foraging task:

```
python run.py +algorithm=ia2c env.name="lbforaging:Foraging-8x8-3p-2f-v3"  
env.time_limit=50 algorithm.total_steps=1000000 algorithm.name=ia2c  
env.standardise_rewards=True algorithm.video_interval=200000 seed=0
```

This trains the IA2C agents, periodically evaluates their performance, and generates videos of rollouts. After training, find the results directory under `outputs/lbforaging:Foraging-8x8-3p-2f-v3/ia2c/...` — logged metrics are stored in `results.csv` and videos in the `videos` directory. Watch some of the videos to see how the agents gradually improve. You can plot the episodic returns throughout training:

```
python utils/postprocessing/plot_runs.py --source outputs
```

Other metrics stored in `results.csv` can be visualised as well, for example the entropy of the policies:

```
python utils/postprocessing/plot_runs.py --source outputs --metric entropy
```

Play with Hyperparameters

Try changing some hyperparameters of the algorithm and see how training is affected — for example the learning rate, the entropy regularisation coefficient, or the network architecture. You can run multiple training runs in parallel using `multiruns`:

```
python run.py -m +algorithm=ia2c env.name="lbforaging:Foraging-8x8-3p-2f-v3"  
env.time_limit=50 algorithm.total_steps=1000000 env.standardise_rewards=True  
seed=0 algorithm.entropy_coef=0.001,0.01,0.1
```

This executes three training runs with different entropy regularisation coefficients in parallel (this should only take a few minutes, depending on the available hardware). All results are stored under `multirun/`. To identify the best hyperparameters or plot the returns of all configurations:

```
python utils/postprocessing/find_best_hyperparams.py --source multirun python
utils/postprocessing/plot_runs.py --source multirun
```

Comparison to Centralised Critics

Next, we compare IA2C to agents trained with the same fundamental algorithm but with centralised critics conditioned on the joint observations of all agents. As a challenging task, we use the robotic warehouse environment. Install it:

```
pip install git+https://github.com/uoe-agents/robotic-warehouse.git
```

Training IA2C and multi-agent A2C (MAA2C) agents in the `rware-tiny-4ag-v2` task would be done as follows:

```
python run.py -m +algorithm=ia2c,maa2c env.name="rware:rware-tiny-4ag-v2"
env.time_limit=500 algorithm.total_steps=20000000
env.standardise_rewards=True seed=0,1,2
```

However, this runs six training runs of 20,000,000 time steps each and might not be feasible. Instead, use the training data already prepared in `deep_marl_data/rware_tiny_4ag`, covering IA2C, MAA2C, IPPO and MAPPO. Compare the algorithms and inspect metrics such as the episodic returns (`mean_episode_returns`), the policy entropy (`entropy`), and the actor (`actor_loss`) and critic (`value_loss`) losses:

```
python utils/postprocessing/plot_runs.py --source
path/to/deep_marl_data/rware_tiny_4ag --metric ...
```

You can also watch the rollout videos stored throughout training in each run's `videos` directory, or load model checkpoints and generate videos of their rollouts:

```
python eval.py path=path/to/deep_marl_data/rware_tiny_4ag/...
env.name="rware:rware-tiny-4ag-v2" env.time_limit=500
```

By default the last checkpoint of the run is loaded; a specific checkpoint can be selected with the `load_step` argument (see the checkpoints directory of a run for the available checkpoints).

Note: only minimal hyperparameter tuning was performed for the training runs in `deep_marl_data`, so the performance and relative ranking of the algorithms might not be reflective of their capabilities under tuned hyperparameters.